StudySync — Peer Matching & Collaborative Study Group Platform

Database Design Specification: 3NF Relational Data Model

Course: Software Engineering & Project Management

Project Group: Group 15 (Section F)

Target Engine: PostgreSQL 14+ Relational Database Management System

Date of Release: October 16, 2026

Design Standard: Third Normal Form (3NF) Relational Architecture with JSONB Extensions

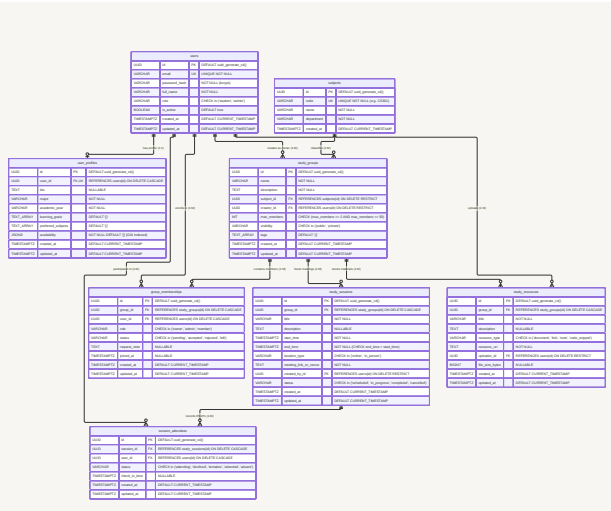
1. Database Architecture & Design Principles

The StudySync persistence tier is built on **PostgreSQL 14+**, chosen for its enterprise-grade ACID transaction guarantees, strict schema constraint enforcement, native JSONB document support, and indexing capabilities (B-Tree and GIN).

Key Architectural Tenets:

1. **Third Normal Form (3NF) Adherence:** Every non-key attribute depends strictly on the primary key, the whole primary key, and nothing but the primary key ($X \to Y$ holds only where X is a superkey). Redundant data duplication is eliminated.
2. **UUID Primary Keys:** All tables utilize globally unique identifiers (`UUIDv4`) generated via the PostgreSQL `uuid-ossp` extension, preventing enumeration attacks and preparing the schema for distributed scaling.
3. **Hybrid Relational-Document Model:** Student recurring weekly availability is modeled as structured `JSONB` with Generalized Inverted Indexing (GIN). This preserves flexible temporal modeling without the performance penalty of creating millions of granular time-slot rows.
4. **Defensive Database Constraints:** Business rules (e.g., maximum member capacity $2 \leq n \leq 50$, positive session durations $\text{end_time} > \text{start_time}$, and enumerated roles) are enforced natively at the database level using `CHECK` constraints.

2. Master Entity-Relationship Diagram (Crow's Foot Notation)



3. Relational Data Dictionary (8 Core Tables)

3.1 `users` (Core Account Credentials & Identity)

Stores authenticated user identities and security credentials.

- **Primary Key:** `id` (UUID)

Column Name	Data Type	Nullable	Constraints & Defaults	Description
`id`	`UUID`	No	`PRIMARY KEY DEFAULT uuid_generate_v4()`	Unique user identifier
`email`	`VARCHAR(255)`	No	`UNIQUE`	Institutional university email address
`password_hash`	`VARCHAR(255)`	No	None	Salted bcrypt cryptographic password hash
`full_name`	`VARCHAR(150)`	No	None	Student full legal name
`role`	`VARCHAR(30)`	No	`DEFAULT 'student' CHECK (role IN ('student', 'admin'))`	System access role
`is_active`	`BOOLEAN`	No	`DEFAULT TRUE`	Soft account activation flag
`created_at`	`TIMESTAMPTZ`	No	`DEFAULT CURRENT_TIMESTAMP`	Account creation timestamp
`updated_at`	`TIMESTAMPTZ`	No	`DEFAULT CURRENT_TIMESTAMP`	Last profile update timestamp

3.2 `subjects` (Standardized Course Catalog)

Central repository of university academic courses and departments.

- **Primary Key:** `id` (UUID)

Column Name	Data Type	Nullable	Constraints & Defaults	Description
`id`	`UUID`	No	`PRIMARY KEY DEFAULT uuid_generate_v4()`	Unique course identifier
`code`	`VARCHAR(50)`	No	`UNIQUE`	Official course code (e.g., `CS301`, `SE202`)
`name`	`VARCHAR(255)`	No	None	Descriptive course title
`department`	`VARCHAR(100)`	Yes	None	Academic department offering course
`created_at`	`TIMESTAMPTZ`	No	`DEFAULT CURRENT_TIMESTAMP`	Record insertion timestamp

3.3 `user_profiles` (Academic Metadata & JSONB Availability)

Stores student academic preferences, learning goals, and weekly recurring free time slots.

- **Primary Key:** `id` (UUID)
- **Foreign Keys:** `user_id` → `users(id)` ON DELETE CASCADE (1-to-1 relationship enforced via `UNIQUE(user_id)`).

Column Name	Data Type	Nullable	Constraints & Defaults	Description
`id`	`UUID`	No	`PRIMARY KEY DEFAULT uuid_generate_v4()`	Unique profile record ID
`user_id`	`UUID`	No	`UNIQUE REFERENCES users(id) ON DELETE CASCADE`	Associated student user ID
`bio`	`TEXT`	Yes	None	Student biographical introduction
`major`	`VARCHAR(150)`	Yes	None	Academic degree program / major
`academic_year`	`VARCHAR(50)`	Yes	None	Cohort level (e.g., `Sophomore`, `Year 3`)
`learning_goals`	`TEXT[]`	No	`DEFAULT '{}'`	Array of student learning objectives
`preferred_subjects`	`TEXT[]`	No	`DEFAULT '{}'`	Array of preferred course subject codes
`availability`	`JSONB`	No	`DEFAULT '[]'` (GIN Indexed)	Structured array of weekly recurring time slots
`created_at`	`TIMESTAMPTZ`	No	`DEFAULT CURRENT_TIMESTAMP`	Creation timestamp
`updated_at`	`TIMESTAMPTZ`	No	`DEFAULT CURRENT_TIMESTAMP`	Modification timestamp

Availability JSONB Schema Example:

```
[
  { "dayOfWeek": 1, "startTime": "09:00", "endTime": "11:30" },
  { "dayOfWeek": 3, "startTime": "14:00", "endTime": "17:00" }
]
```

3.4 `study_groups` (Study Groups & Capacity Bounds)

Stores collaborative student study groups linked to course subjects.

- **Primary Key:** `id` (UUID)
- **Foreign Keys:**
  - `subject_id` → `subjects(id)` ON DELETE RESTRICT
  - `creator_id` → `users(id)` ON DELETE RESTRICT

Column Name	Data Type	Nullable	Constraints & Defaults	Description
`id`	`UUID`	No	`PRIMARY KEY DEFAULT uuid_generate_v4()`	Unique group identifier
`name`	`VARCHAR(200)`	No	None	Display title of the study group
`description`	`TEXT`	No	None	Detailed purpose and topics covered
`subject_id`	`UUID`	No	`REFERENCES subjects(id) ON DELETE RESTRICT`	Associated university course
`creator_id`	`UUID`	No	`REFERENCES users(id) ON DELETE RESTRICT`	User who created and owns the group
`max_members`	`INT`	No	`DEFAULT 8 CHECK (max_members >= 2 AND max_members <= 50)`	Enforced member capacity ceiling
`visibility`	`VARCHAR(20)`	No	`DEFAULT 'public' CHECK (visibility IN ('public', 'private'))`	Directory discoverability status
`tags`	`TEXT[]`	No	`DEFAULT '{}'`	Searchable topic tags
`created_at`	`TIMESTAMPTZ`	No	`DEFAULT CURRENT_TIMESTAMP`	Group creation timestamp
`updated_at`	`TIMESTAMPTZ`	No	`DEFAULT CURRENT_TIMESTAMP`	Group update timestamp

3.5 `group_memberships` (Roster Enrollment & Approval Workflow)

Manages membership applications, approval statuses, and group roles.

- **Primary Key:** `id` (UUID)
- **Foreign Keys:**
  - `group_id` → `study_groups(id)` ON DELETE CASCADE
  - `user_id` → `users(id)` ON DELETE CASCADE
- **Unique Constraint:** `CONSTRAINT uq_group_user UNIQUE(group_id, user_id)` (Guarantees at most one enrollment record per student per group).

Column Name	Data Type	Nullable	Constraints & Defaults	Description
`id`	`UUID`	No	`PRIMARY KEY DEFAULT uuid_generate_v4()`	Unique membership record ID
`group_id`	`UUID`	No	`REFERENCES study_groups(id) ON DELETE CASCADE`	Enrolled study group
`user_id`	`UUID`	No	`REFERENCES users(id) ON DELETE CASCADE`	Enrolled student user
`role`	`VARCHAR(30)`	No	`DEFAULT 'member' CHECK (role IN ('owner', 'admin', 'member'))`	Group administrative privilege
`status`	`VARCHAR(30)`	No	`DEFAULT 'pending' CHECK (status IN ('pending', 'accepted', 'rejected', 'left'))`	Application review state

Column Name	Data Type	Nullable	Constraints & Defaults	Description
`request_note`	`TEXT`	Yes	None	Introductory motivation message from applicant
`joined_at`	`TIMESTAMPTZ`	Yes	None	Timestamp when status changed to `'accepted'`
`created_at`	`TIMESTAMPTZ`	No	`DEFAULT CURRENT_TIMESTAMP`	Submission timestamp
`updated_at`	`TIMESTAMPTZ`	No	`DEFAULT CURRENT_TIMESTAMP`	Decision timestamp

3.6 `study_sessions` (Scheduled Meetings & Venues)

Stores scheduled group study meetings with location and time bounds.

- **Primary Key:** `id` (UUID)
- **Foreign Keys:**
  - `group_id` → `study_groups(id)` ON DELETE CASCADE
  - `created_by_id` → `users(id)` ON DELETE RESTRICT
- **Check Constraint:** `CONSTRAINT chk_session_time_interval CHECK (end_time > start_time)`

Column Name	Data Type	Nullable	Constraints & Defaults	Description
`id`	`UUID`	No	`PRIMARY KEY DEFAULT uuid_generate_v4()`	Unique session identifier
`group_id`	`UUID`	No	`REFERENCES study_groups(id) ON DELETE CASCADE`	Host study group
`title`	`VARCHAR(200)`	No	None	Session title / topic
`description`	`TEXT`	Yes	None	Meeting agenda and preparation instructions
`start_time`	`TIMESTAMPTZ`	No	None	Scheduled session start timestamp
`end_time`	`TIMESTAMPTZ`	No	`CHECK (end_time > start_time)`	Scheduled session end timestamp
`location_type`	`VARCHAR(30)`	No	`DEFAULT 'online' CHECK (location_type IN ('online', 'in_person'))`	Delivery medium
`meeting_link_or_venue`	`TEXT`	No	None	Virtual URL (Zoom/Meet) or physical room code
`created_by_id`	`UUID`	No	`REFERENCES users(id) ON DELETE RESTRICT`	Organizer who scheduled session
`status`	`VARCHAR(30)`	No	`DEFAULT 'scheduled' CHECK (status IN ('scheduled', 'in_progress', 'completed', 'cancelled'))`	Lifecycle state
`created_at`	`TIMESTAMPTZ`	No	`DEFAULT CURRENT_TIMESTAMP`	Record timestamp
`updated_at`	`TIMESTAMPTZ`	No	`DEFAULT CURRENT_TIMESTAMP`	Last change timestamp

3.7 `session_attendees` (RSVP & Verified Attendance Check-In)

Tracks participant attendance commitments and verified physical/virtual check-ins.

- **Primary Key:** `id` (UUID)
- **Foreign Keys:**
  - `session_id` → `study_sessions(id)` ON DELETE CASCADE
  - `user_id` → `users(id)` ON DELETE CASCADE
- **Unique Constraint:** `CONSTRAINT uq_session_user UNIQUE(session_id, user_id)`

Column Name	Data Type	Nullable	Constraints & Defaults	Description
`id`	`UUID`	No	`PRIMARY KEY DEFAULT uuid_generate_v4()`	Unique attendee record ID
`session_id`	`UUID`	No	`REFERENCES study_sessions(id) ON DELETE CASCADE`	Associated meeting
`user_id`	`UUID`	No	`REFERENCES users(id) ON DELETE CASCADE`	Invited group member
`status`	`VARCHAR(30)`	No	`DEFAULT 'tentative' CHECK (status IN ('attending', 'declined', 'tentative', 'attended', 'absent'))`	RSVP and attendance state
`check_in_time`	`TIMESTAMPTZ`	Yes	None	Timestamp when attendance was checked in
`created_at`	`TIMESTAMPTZ`	No	`DEFAULT CURRENT_TIMESTAMP`	Invitation timestamp
`updated_at`	`TIMESTAMPTZ`	No	`DEFAULT CURRENT_TIMESTAMP`	Status change timestamp

3.8 `study_resources` (Access-Controlled Academic Materials)

Stores shared notes, syllabi links, and code snippets categorized by study group.

- **Primary Key:** `id` (UUID)
- **Foreign Keys:**
  - `group_id` → `study_groups(id)` ON DELETE CASCADE
  - `uploader_id` → `users(id)` ON DELETE RESTRICT

Column Name	Data Type	Nullable	Constraints & Defaults	Description
`id`	`UUID`	No	`PRIMARY KEY DEFAULT uuid_generate_v4()`	Unique resource identifier
`group_id`	`UUID`	No	`REFERENCES study_groups(id) ON DELETE CASCADE`	Parent study group
`title`	`VARCHAR(200)`	No	None	Display title of the material
`description`	`TEXT`	Yes	None	Resource context or notes
`resource_type`	`VARCHAR(30)`	No	`CHECK (resource_type IN ('document', 'link', 'note', 'code_snippet'))`	Format category
`resource_uri`	`TEXT`	No	None	External URL or secured storage path
`uploader_id`	`UUID`	No	`REFERENCES users(id) ON DELETE RESTRICT`	Member who shared resource
`file_size_bytes`	`BIGINT`	Yes	None	Document file size (if binary)
`created_at`	`TIMESTAMPTZ`	No	`DEFAULT CURRENT_TIMESTAMP`	Upload timestamp
`updated_at`	`TIMESTAMPTZ`	No	`DEFAULT CURRENT_TIMESTAMP`	Modification timestamp

4. Third Normal Form (3NF) Compliance Analysis

The schema satisfies all relational normalization criteria:

1. **First Normal Form (1NF):**
   - Each table has a defined primary key (`id` UUID).
   - All columns store atomic values relative to the relational domain (atomic scalar values or strongly typed composite JSONB objects).
   - There are no repeating column groups (e.g., `slot1`, `slot2`, `slot3`).
2. **Second Normal Form (2NF):**
   - The schema is in 1NF.

- All primary keys are single-column UUIDs. Consequently, no non-prime attribute can be partially functionally dependent on a composite candidate key.

3. **Third Normal Form (3NF):**
   - The schema is in 2NF.
   - No non-prime attribute is transitively dependent on the primary key ($X \to Y \to Z$).
   - For example: Course information (`code`, `name`, `department`) is not embedded inside `study_groups`; instead, `study_groups` references `subjects(id)` via foreign key, preventing update and insertion anomalies.
   - Student academic metadata (`major`, `bio`, `availability`) is segregated into `user_profiles` rather than cluttering core authentication records in `users`.

---

5. Indexing & Query Optimization Strategy

```sql
-- 1. Authentication Index
CREATE INDEX idx_users_email ON users(email);

-- 2. Course Catalog Index
CREATE INDEX idx_subjects_code ON subjects(code);

-- 3. Temporal Availability Index (Generalized Inverted Index for JSONB)
CREATE INDEX idx_user_profiles_user_id ON user_profiles(user_id);
CREATE INDEX idx_user_profiles_availability ON user_profiles USING GIN (availability);

-- 4. Group Catalog Search Indexes
CREATE INDEX idx_study_groups_subject_id ON study_groups(subject_id);
CREATE INDEX idx_study_groups_creator_id ON study_groups(creator_id);
CREATE INDEX idx_study_groups_visibility ON study_groups(visibility);

-- 5. Membership Resolution Indexes
CREATE INDEX idx_group_memberships_group_id ON group_memberships(group_id);
CREATE INDEX idx_group_memberships_user_id ON group_memberships(user_id);
CREATE INDEX idx_group_memberships_status ON group_memberships(status);

-- 6. Calendar Range Query Index (Composite B-Tree)
CREATE INDEX idx_study_sessions_group_id ON study_sessions(group_id);
CREATE INDEX idx_study_sessions_time_range ON study_sessions(start_time, end_time);

-- 7. Attendee Lookup Indexes
CREATE INDEX idx_session_attendees_session_id ON session_attendees(session_id);
CREATE INDEX idx_session_attendees_user_id ON session_attendees(user_id);

-- 8. Resource Access Indexes
CREATE INDEX idx_study_resources_group_id ON study_resources(group_id);
CREATE INDEX idx_study_resources_uploader_id ON study_resources(uploader_id);
```

Optimization Benefits:

- **Sub-15ms Catalog Queries:** Filtering public groups by course code (`idx_study_groups_subject_id`) and status (`idx_study_groups_visibility`) utilizes direct index scans.
- **Fast Interval Overlap Queries:** The composite index on `(start_time, end_time)` accelerates collision checking across hundreds of member sessions into sub-millisecond range queries.
- **Fast Schedule Intersection:** The GIN index on `user_profiles.availability` enables PostgreSQL's native JSONB containment operators (`@>`, `?&`) to filter matching time slots efficiently.

---

6. Concurrency Control, Row Locking & Transaction Boundaries

The Group Capacity Race Condition Problem:

If a study group has 1 open seat (`accepted_count = 5`, `max_members = 6`), two group administrators reviewing applications concurrently could approve two applicants simultaneously, leading to 7 members and breaching the capacity constraint.

Solution: Row-Level Locking in Atomic Transactions

StudySync mitigates this race condition in `GroupRepository.updateMembershipStatus` using PostgreSQL's transactional row-level lock:

```
BEGIN;

-- 1. Exclusively lock the study group record for the duration of the transaction
SELECT id, max_members
FROM study_groups
WHERE id = $1
FOR UPDATE;

-- 2. Count current active accepted members
SELECT COUNT(*) AS active_count
FROM group_memberships
WHERE group_id = $1 AND status = 'accepted';

-- 3. If active_count >= max_members, abort transaction
-- Return HTTP 400 Bad Request ('GROUP_CAPACITY_REACHED')

-- 4. Otherwise, update membership to accepted
UPDATE group_memberships
SET status = 'accepted', joined_at = CURRENT_TIMESTAMP, updated_at = CURRENT_TIMESTAMP
WHERE group_id = $1 AND user_id = $2;

COMMIT;
```

7. Referential Integrity & Cascading Architecture

Relationship	Cascade Rule	Business Rationale
`users` → `user_profiles`	`ON DELETE CASCADE`	A profile cannot exist without its owning user.
`users` → `group_memberships`	`ON DELETE CASCADE`	Deleting an account removes all associated memberships and applications.
`study_groups` → `group_memberships`	`ON DELETE CASCADE`	Disbanding a study group automatically dissolves all member enrollments.
`study_groups` → `study_sessions`	`ON DELETE CASCADE`	Removing a study group deletes all associated calendar meetings.
`study_sessions` → `session_attendees`	`ON DELETE CASCADE`	Canceling a session automatically cleans up all associated attendee RSVPs.
`study_groups` → `study_resources`	`ON DELETE CASCADE`	Disbanding a study group purges all group-scoped study documents and links.
`subjects` → `study_groups`	`ON DELETE RESTRICT`	An active academic subject cannot be deleted if active study groups reference it.
`users` → `study_groups` (creator)	`ON DELETE RESTRICT`	Group ownership must be explicitly reassigned before a creator account can be removed.